



République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique

Département IA&SD

Spécialité : SII

Méta-heuristiques et Algorithmes Evolutionnaires

Problème du Sacs à Dos Multiples

Rapport de Projet

Présenté par : Merabti Amar

Présenté à : Naila Houacine

Le 06/04/2024

Master 1 SII 2023/24

1. Introduction

Le problème du sac à dos multiple (Multiple Knapsack Problem - MKP) est un problème d'optimisation combinatoire largement étudié dans le domaine de l'informatique et de l'optimisation. Il consiste à répartir un ensemble d'objets donnés dans plusieurs sacs à dos de capacités différentes, tout en maximisant la valeur totale des objets placés dans les sacs et en respectant les contraintes de poids imposées par chaque sac. Ce problème présente de nombreuses applications pratiques dans des domaines tels que la logistique, la gestion des ressources, la planification de production, etc.

Dans le cadre de cette première partie du projet, nous nous sommes penchés sur la résolution du problème des Sacs à Dos Multiples en utilisant différentes approches algorithmiques basées espace d'états. Notre objectif était de modéliser le problème, de le résoudre en utilisant des méthodes exactes telles que la recherche en profondeur (DFS) et la recherche en largeur (BFS), ainsi que d'explorer l'approche heuristique A* pour trouver des solutions efficaces.

Ce rapport détaille notre démarche pour modéliser et résoudre le problème du sac à dos multiple, ainsi que les résultats obtenus à travers des expérimentations réalisées sur différentes tailles de problème. Nous présenterons également une comparaison des performances des différentes approches utilisées, en mettant en évidence les avantages et les limites de chacune.

Dans la suite de ce rapport, nous commencerons par définir le problème du sac à dos multiple, suivi d'une explication détaillée des méthodes utilisées pour le résoudre, puis nous présenterons les résultats de nos expériences et enfin nous conclurons sur nos observations et perspectives.

2. Définition du Problème

Le problème des Sacs à Dos Multiples (MKP) est une variante du célèbre problème du sac à dos (Knapsack Problem), qui implique la répartition d'un ensemble d'objets dans plusieurs sacs à dos, chacun ayant sa propre capacité maximale. Chaque objet possède une valeur spécifique et un poids déterminé, et l'objectif est de sélectionner les objets à placer dans les sacs de manière à maximiser la valeur totale des objets tout en respectant les contraintes de capacité des sacs.

Formellement, soit n le nombre d'objets disponibles et m le nombre de sacs à dos, le problème des Sacs à Dos Multiples peut être défini de la manière suivante :

Données :

- n objets, où chaque objet i est caractérisé par sa valeur v_i et son poids p_i .
- m sacs à dos, avec une capacité maximale C_j pour le sac j , où $j = 1, 2, \dots, m$.

Objectif :

- Sélectionner un ensemble d'objets à placer dans les sacs de manière à maximiser la somme des valeurs des objets sélectionnés, tout en respectant les contraintes de capacité de chaque sac.

Contraintes :

- Chaque objet peut être placé dans un seul sac à dos ou être laissé de côté.
- La somme des poids des objets placés dans chaque sac ne doit pas dépasser la capacité maximale du sac correspondant.

Considérant un exemple illustratif :

Nombre d'objets = 7

Nombre de sacs = 5

Les caractéristiques des objets sont données par la matrice suivante :

$$\begin{pmatrix} 10 & 11 & 13 & 7 & 13 & 7 & 14 \\ 8 & 19 & 18 & 4 & 8 & 7 & 6 \end{pmatrix}$$

Où la première ligne représente les valeurs des objets et la deuxième ligne représente leurs poids respectifs.

Les capacités maximales des sacs à dos sont définies comme suit :

$$(10 \ 15 \ 14 \ 15 \ 13)$$

Dans ce cas, le problème consiste à déterminer quels objets placer dans chaque sac de manière à maximiser la valeur totale tout en respectant les capacités maximales des sacs à dos.

Exemple d'une solution valide :

Objet 1	Objet 2	Objet 3	Objet 4	Objet 5	Objet 6	Objet 7
Sac 1	-1	-1	Sac 2	Sac 3	Sac 5	Sac 4

Où **-1** veut dire que l'objet n'est placé dans un sac (laissé de côté).

Dans Cette solution, il n'y a pas de dépassement des capacités maximales des sac, pour chaque sac le cumul des poids des objets placés dans ce sac est inférieur ou égale à la capacité maximale du sac, donc elle respecte les contraintes citées précédemment.

Exemple d'une solution non valide :

Objet 1	Objet 2	Objet 3	Objet 4	Objet 5	Objet 6	Objet 7
Sac 1	-1	Sac 1	Sac 2	Sac 3	Sac 5	Sac 4

Dans cette solution on remarque que les objet 1 et 3 sont placé dans le sac 1, le cumul de leurs poids est : $8 + 18 = 26$ qui est supérieur à la capacité maximale du sac 1 (10), donc les contraintes ne sont pas respectées et la solution n'est pas valide.

Dans la suite de ce rapport, nous explorerons différentes approches pour résoudre efficacement le problème des Sacs à Dos Multiples, en utilisant des méthodes exactes telles que la recherche en profondeur et la recherche en largeur, ainsi qu'une approche heuristique basée sur l'algorithme A*.

3. Fonctionnement et Algorithme de Chaque Approche

Dans cette section, nous allons expliquer en détail le fonctionnement de trois approches différentes pour résoudre le problème des Sacs à Dos Multiples : la recherche en profondeur (DFS), la recherche en largeur (BFS) et l'algorithme A*.

3.1. Recherche en Profondeur (DFS)

La recherche en profondeur est une méthode de parcours d'arbre où l'algorithme explore le plus profondément possible le premier chemin avant de revenir en arrière et d'explorer les autres chemins. Dans le contexte du problème des Sacs à Dos Multiples, l'algorithme DFS procède comme suit :

Initialisation : Commence par un état initial où aucun objet n'est placé dans aucun sac.

Exploration des Successeurs : À chaque étape, l'algorithme explore un nœud en plaçant un objet dans un sac disponible, puis passe au nœud suivant en explorant tous les successeurs possibles en profondeur d'abord, en respectant toujours les contraintes des capacités maximales des sacs.

Retour en Arrière : Si l'algorithme atteint une feuille de l'arbre de recherche ou si aucune amélioration n'est possible, il revient en arrière pour explorer d'autres options.

Critère d'Arrêt : L'algorithme s'arrête lorsque tous les nœuds ont été explorés ou qu'une solution optimale est trouvée.

L'algorithme DFS peut être implémenté de manière récursive ou itérative, en fonction des préférences de programmation.

Dans notre implémentation on a utilisé la version itérative en utilisant une pile pour gérer d'une manière LIFO les nœuds à explorer afin d'assurer le parcours en profondeur d'abord.

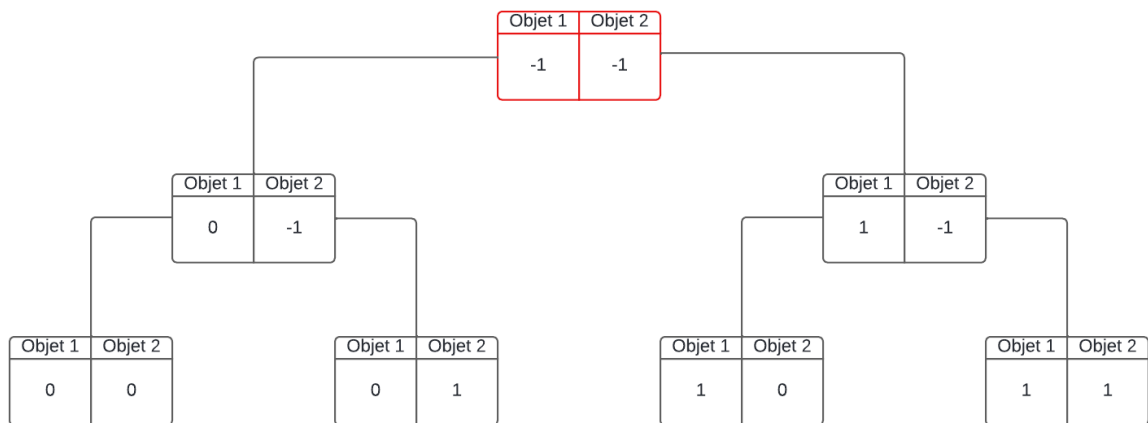
Complexité :

- Dans le pire des cas, la complexité temporelle de DFS est de $O(b^d)$, où b est le facteur de branchement de l'arbre de recherche (le nombre moyen de successeurs par nœud) et d est la profondeur maximale de l'arbre.
- Cette complexité temporelle dépend de la taille de l'arbre de recherche et de la profondeur maximale. Dans le pire des cas, DFS peut potentiellement explorer tout l'arbre en profondeur avant de trouver une solution ou d'atteindre une feuille de l'arbre.

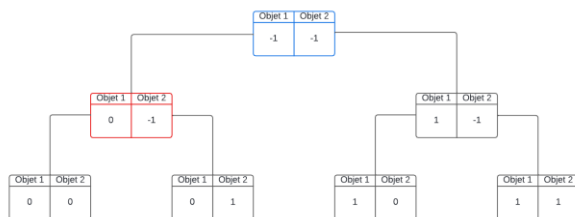
Exemple illustratif :

Considérant cet exemple illustratif du fonctionnement de l'algorithme DFS, où on a deux objets et deux sacs :

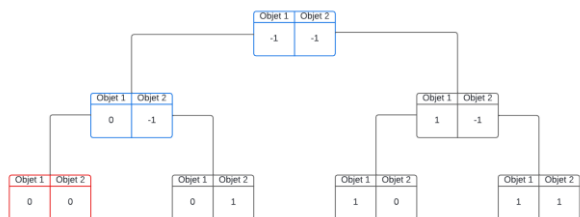
Itération 0



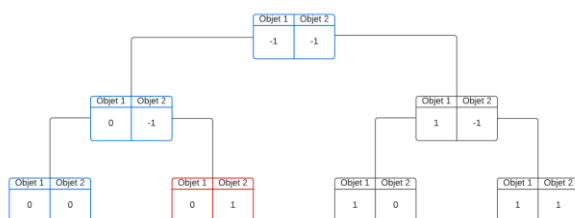
Itération 1



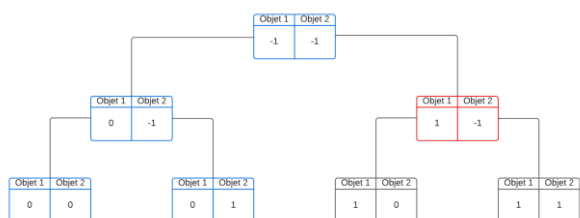
Itération 2



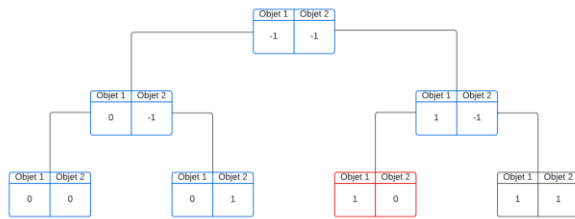
Itération 3



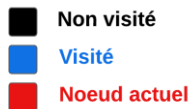
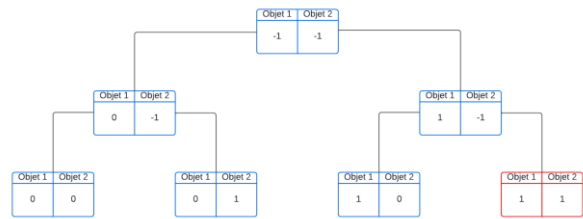
Itération 4



Itération 5



Itération 6



3.2. Recherche en Largeur (BFS)

La recherche en largeur est une méthode de parcours d'arbre où l'algorithme explore tous les nœuds d'un niveau avant de passer au niveau suivant. Dans le contexte du problème des Sacs à Dos Multiples, l'algorithme BFS procède comme suit :

Initialisation : Commence par un état initial où aucun objet n'est placé dans aucun sac.

Exploration des Successeurs : À chaque étape, l'algorithme explore tous les successeurs possibles du nœud actuel en largeur d'abord en plaçant un objet dans un sac disponible toute en respectant les contraintes des capacités maximales.

Nouveau Niveau : Une fois tous les successeurs d'un niveau explorés, l'algorithme passe au niveau suivant et explore les successeurs de ces nœuds.

Critère d'Arrêt : L'algorithme s'arrête lorsqu'il a exploré tous les nœuds de l'arbre ou qu'une solution optimale est trouvée.

L'algorithme BFS peut être implémenté de manière récursive ou itérative, en fonction des préférences de programmation.

Dans notre implémentation on a utilisé la version itérative en utilisant une file pour gérer d'une manière FIFO les nœuds à explorer afin d'assurer le parcours en largeurs d'abord.

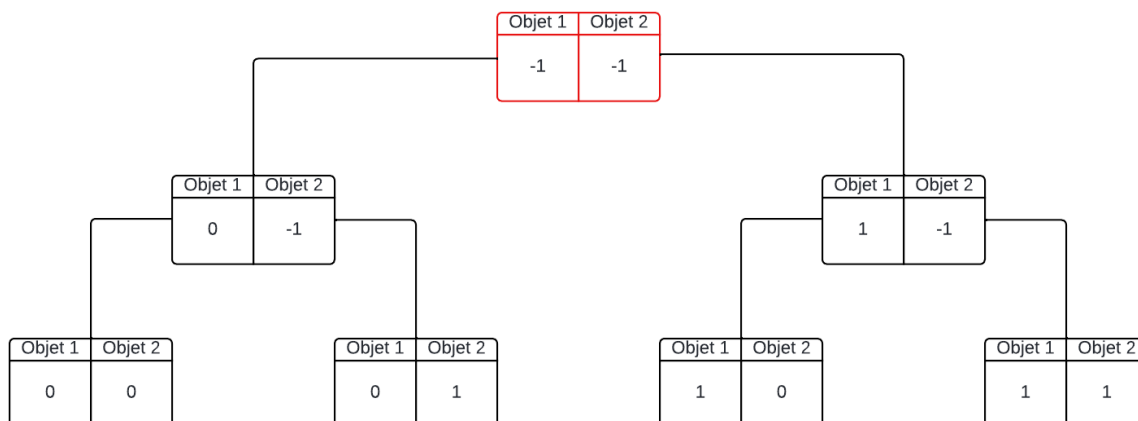
Complexité :

- La complexité temporelle de BFS est également de $O(b^d)$, où b est le facteur de branchement de l'arbre de recherche et d est la profondeur maximale de l'arbre.
- Cependant, BFS explore d'abord tous les nœuds à une profondeur donnée avant de passer à la profondeur suivante. Cela signifie que BFS pourrait potentiellement explorer plus de nœuds que DFS dans certains cas, mais il est garanti de trouver la solution la plus profonde.

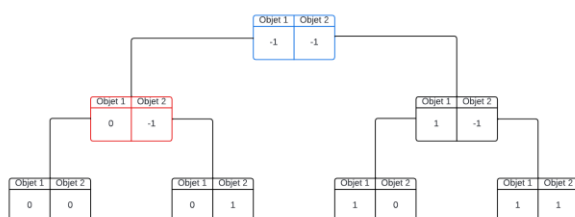
Exemple illustratif :

Considérant cet exemple illustratif du fonctionnement de l'algorithme BFS, où on a deux objets et deux sacs :

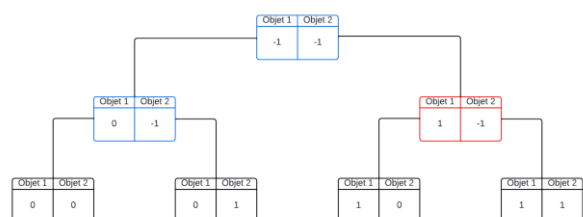
Itération 0



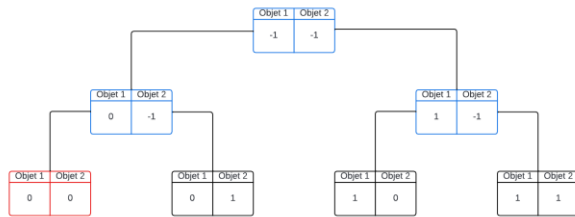
Itération 1



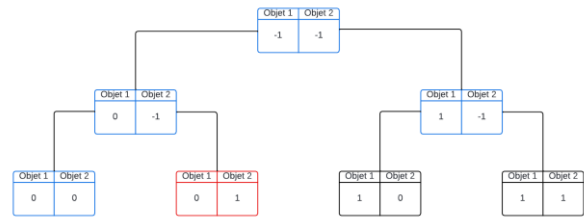
Itération 2



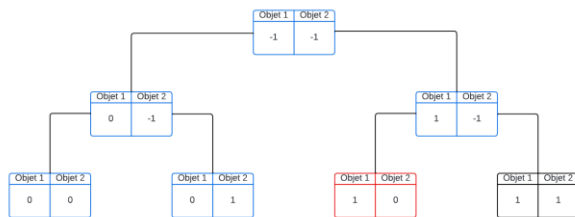
Itération 3



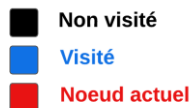
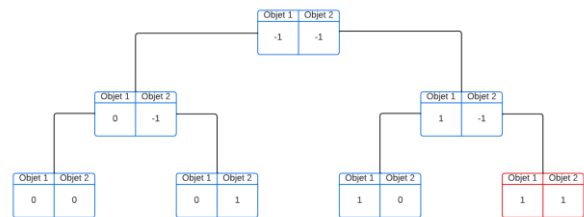
Itération 4



Itération 5



Itération 6



3.3. Algorithme A*

L'algorithme A* est une méthode de recherche heuristique qui combine les avantages de la recherche en profondeur et de la recherche en largeur en utilisant une fonction d'évaluation pour guider la recherche vers la solution optimale. Dans le contexte du problème des Sacs à Dos Multiples, l'algorithme A* procède comme suit :

Initialisation : Commence par un état initial où aucun objet n'est placé dans aucun sac.

Évaluation des Successeurs : À chaque étape, l'algorithme évalue tous les successeurs possibles du nœud actuel en fonction d'une fonction d'évaluation combinant le coût réel et une estimation heuristique du coût restant.

Exploration : L'algorithme explore d'abord les nœuds les moins coûteux en utilisant la fonction d'évaluation.

Critère d'Arrêt : L'algorithme s'arrête lorsqu'il atteint un état, final cela signifierait avoir trouvé une solution satisfaisante qui remplit les sacs selon les contraintes données.

Complexité :

- La complexité temporelle d'A* dépend de plusieurs facteurs, notamment la qualité de l'heuristique utilisée, le facteur de branchement de l'arbre de recherche, la profondeur de la solution optimale et la manière dont l'algorithme est implémenté.
- En général, la complexité temporelle d'A* peut être exprimée comme $O(b^d)$, mais avec une réduction significative du nombre de nœuds explorés grâce à l'utilisation d'une heuristique admissible et cohérente.
- L'efficacité d'A* dépend donc fortement du choix de l'heuristique. Dans le meilleur des cas, avec une heuristique admissible et cohérente, A* peut explorer un nombre minimal de nœuds pour trouver la solution optimale. Cependant, dans le pire des cas, il peut être aussi inefficace que DFS ou BFS.

L'algorithme A* nécessite une fonction de cout et une fonction heuristique.

L'algorithme A* nécessite une heuristique admissible pour estimer le coût restant jusqu'à la solution optimale. Dans le contexte du problème des Sacs à Dos Multiples, différentes heuristiques peuvent être utilisées pour guider la recherche vers une solution efficace.

3.3.1. Fonction Cout pour A*

Dans notre implémentation on a utilisé la somme des poids des objets qui sont déjà placés dans un sac comme une fonction cout.

3.3.2. Heuristiques pour l'Algorithme A*

Dans le cadre de l'algorithme A*, on a implémenté trois heuristiques pour guider la recherche vers une solution optimale ou proche de l'optimale pour le problème des Sacs à Dos Multiples :

Heuristique 1 : Cette heuristique est basée sur le rapport entre les valeurs des objets non sélectionnés et les capacités restantes des sacs à dos. Elle vise à maximiser ce rapport pour orienter la recherche vers les solutions les plus prometteuses.

Heuristique 2 : Cette heuristique utilise une relaxation linéaire du problème pour estimer le coût restant jusqu'à la solution optimale. Elle consiste à maximiser la valeur totale des objets qui peuvent être ajoutés aux sacs à dos en respectant les capacités restantes.

Dans le contexte du problème du sac à dos, la relaxation linéaire consiste à supposer que les objets peuvent être fractionnés ou découpés en parties plus petites. Concrètement, cela signifie que vous ne nous limitons pas à placer un objet entier dans un sac à dos, mais que vous pouvez en placer une fraction.

Heuristique 3 : Cette heuristique vise à équilibrer la valeur totale des objets sélectionnés et la répartition équitable des poids entre les sacs à dos. Elle pénalise les écarts importants entre les poids des sacs à dos pour favoriser une répartition équilibrée des objets.

Ces heuristiques sont utilisées dans l'algorithme A* pour guider la recherche vers des solutions efficaces pour le problème des Sacs à Dos Multiples.

A noter que chaque heuristique a ces avantages et ces inconvénients.

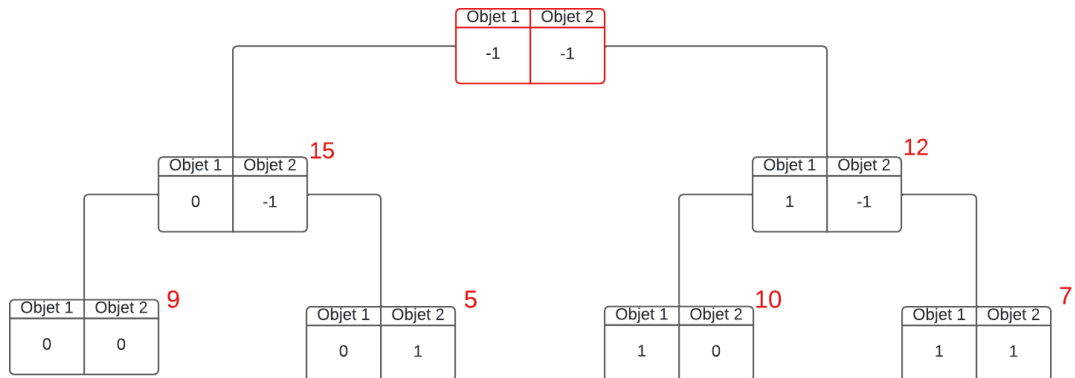
Voici un tableau comparatif des trois heuristiques :

Heuristique	Avantages	Inconvénients
Heuristique 1	- Facile à mettre en œuvre. - Rapide à calculer. - Donne une idée de la valeur potentielle de la solution.	- Peut ne pas être très précise, en particulier pour les problèmes complexes. - Ne prend pas en compte les contraintes de capacité des sacs dans le calcul de l'estimation.
Heuristique 2	- Tient compte des capacités restantes des sacs, ce qui permet d'évaluer plus précisément la faisabilité de la solution. - Prend en compte la structure du problème et les contraintes de capacité.	- Plus complexe à mettre en œuvre que l'heuristique 1. - Peut nécessiter plus de temps de calcul en raison de sa complexité.
Heuristique 3	- Tient compte de l'équilibre des poids entre les sacs, ce qui peut conduire à des solutions plus équilibrées. - Prend en compte la valeur totale des objets ainsi que la répartition des poids.	- Peut être difficile à ajuster pour obtenir un bon équilibre entre la valeur totale et la répartition des poids. - Peut nécessiter une connaissance approfondie du problème pour ajuster les paramètres correctement.

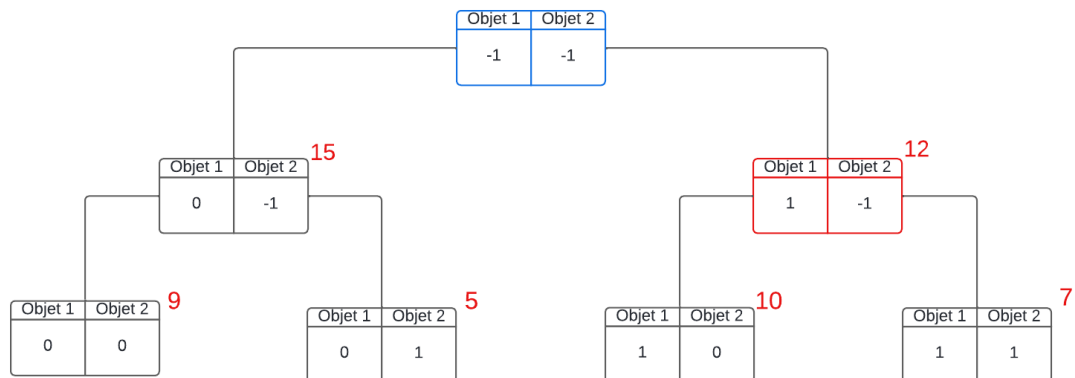
Exemple illustratif :

Considérant cet exemple illustratif du fonctionnement de l'algorithme A*, où on a deux objets et deux sacs :

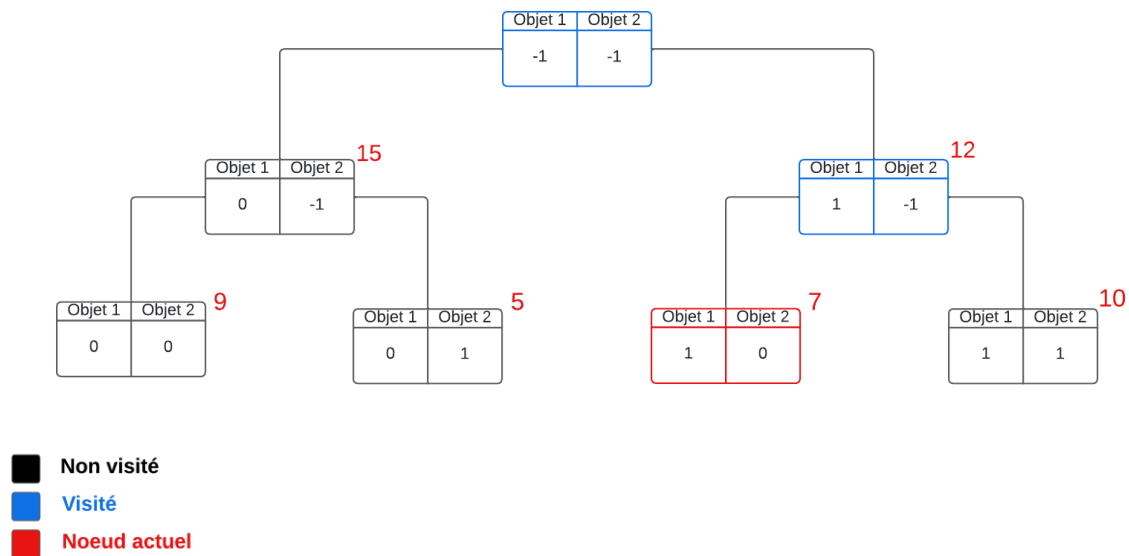
Itération 0



Itération 1



Itération 2



La valeur en rouge est la valeur du cout qui combine le cout réel et la valeur de la fonction heuristique.

On remarque A* développe moins de nœuds que DFS et BFS, mais la solution trouvée peut être non optimal, dans l'exemple précédent on remarque qu'il existe une solution meilleure avec un cout total meilleur, cependant A* a convergé vers une autre solution, mais ça reste proche de l'optimalité.

4. Expérimentation des Algorithmes

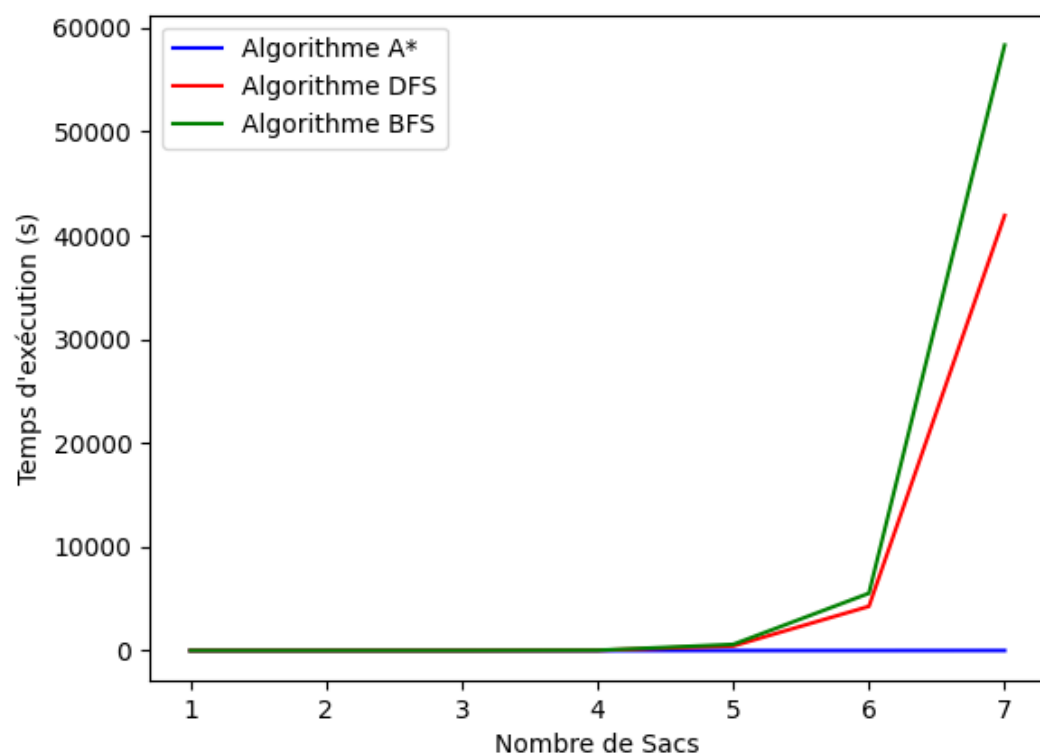
Dans cette section, nous présentons les résultats obtenus lors de l'exécution des algorithmes DFS, BFS et A* sur des instances de différentes tailles du problème. Nous mesurons le temps d'exécution et le nombre de nœuds développés pour chaque approche afin de comparer leurs performances.

On a fixé le nombre d'objets à 7, et à chaque itération on augmente le nombre de sac pour un maximum 7 objets, 7 sacs.

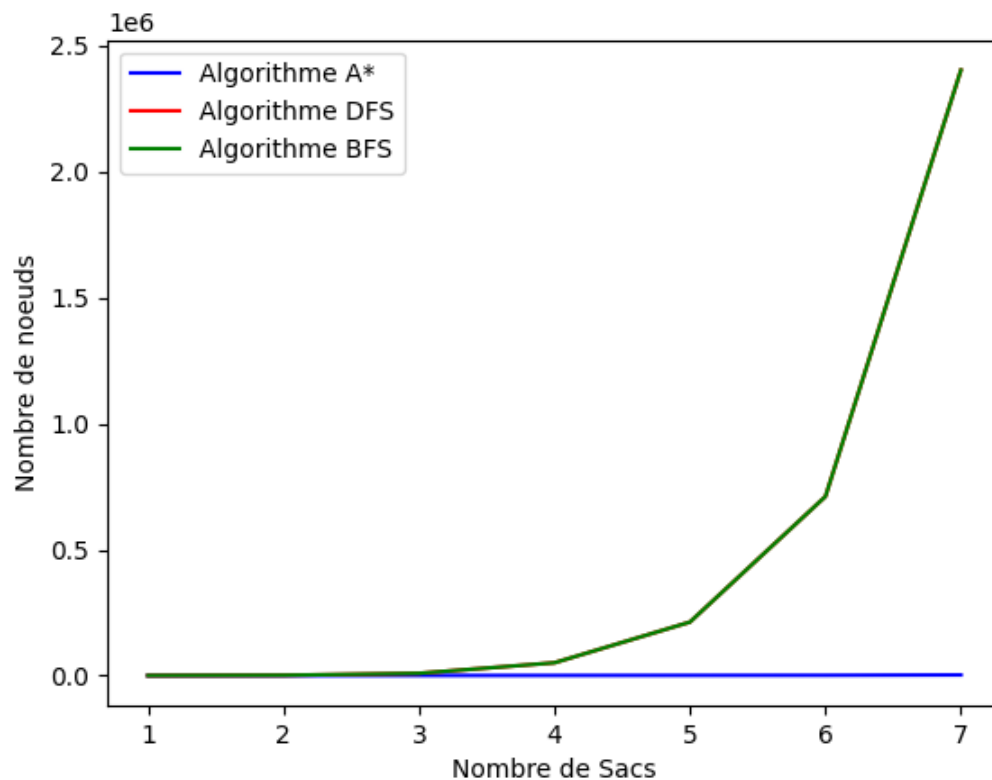
Pour l'algorithme A* on a choisi l'heuristique 2 (la relaxation linéaire).

Les résultats sont résumés dans le tableau suivant et les graphes suivants :

Nombre d'Objets	Nombre de Sacs	Nombre de Nœuds Générés			Temps d'Exécution (secondes)		
		DFS	BFS	A*	DFS	BFS	A*
7	1	28	28	2	0.0023393	0.0027013	0.004346
7	2	1136	1136	70	0.0260099	0.0335277	0.008117
7	3	8826	8826	372	0.6246533	0.5050211	0.017885
7	4	50770	50770	522	22.050239	26.844015	0.019951
7	5	212380	212380	825	439.77689	594.16925	0.094824
7	6	710867	710867	1150	4258.9124	5517.5422	0.160211
7	7	2403898	2403898	2508	41925.569	58345.671	3.462836



[Temps d'exécution de chaque algorithme](#)



[Nombre de nœuds développés par chaque algorithme](#)

5. Analyse et Comparaison des Résultats

5.1. Temps d'Exécution :

- En général, on observe une tendance à une augmentation du temps d'exécution avec l'augmentation du nombre de sacs pour toutes les approches.
- Les algorithmes BFS et DFS montrent une croissance exponentielle du temps d'exécution à mesure que le nombre de sacs augmente. Cela est dû à leur nature exploratoire et à la recherche en largeur et profondeur respectivement.
- Les temps d'exécution des algorithmes DFS et BFS sont énormément grands, ils atteignent les environs de 60000 secondes $\approx$ 16 heures, bien que la taille du problème ne dépasse pas les 7 objets et les 7 sacs, c'est énorme, cela est directement liée aux complexités exponentielles des deux algorithmes.
- La légère différence entre les temps d'exécution des deux algorithmes DFS et BFS peut être dû à l'implémentation elle-même, le fait que l'un utilise une pile et l'autre utilise une file, et aussi le fait que notre implémentation assure la non-répétition des solutions générées (successeurs), cette opération peut être computationnellement lourde et dépend de la structure de données elle-même.

- L'algorithme A* quant à lui, montre le meilleur temps d'exécution parmi les trois approches pour toutes les instances de problème testées. Cela est dû à l'utilisation d'une heuristique efficace (heuristique 2) qui guide la recherche vers des solutions prometteuses tout en évitant de parcourir inutilement de nombreux nœuds.

5.1. Nombre de Nœuds Générés :

- Le nombre de nœuds générés est également un indicateur important de la performance des algorithmes de recherche.
- Les algorithmes DFS et BFS génèrent les mêmes nombres de nœuds pour chaque instance testée. Cela est dû au fait qu'ils explorent exhaustivement toutes les possibilités que ce soit en largeur (BFS) ou en profondeur (DFS).
- Le nombre de nœuds généré par DFS et BFS augmente d'une façon exponentielle, malgré -encore une fois- que la taille du problème est relativement petite, c'est la complexité exponentielle.
- L'algorithme A* génère le nombre le plus faible de nœuds parmi les trois approches pour toutes les instances de problème testées. Cela est dû à l'utilisation d'une heuristique (heuristique 2) efficace qui guide la recherche vers des solutions prometteuses dès le début.

6. Conclusion

L'analyse comparative des trois algorithmes montre clairement que l'algorithme A^* avec l'heuristique 2 qui utilise la relaxation linéaire est le plus performant en termes de temps d'exécution et de nombre de nœuds générés. Cela est dû à sa capacité à guider efficacement la recherche vers des solutions prometteuses tout en évitant l'exploration exhaustive de l'espace de recherche. L'efficacité de l'heuristique utilisée dans A^* joue un rôle crucial dans cette performance, permettant de trouver des solutions optimales ou proches de l'optimalité dans un temps raisonnable, même pour des instances de problème relativement complexes. Cependant, A^* ne nous mène pas toujours à la solution optimale, il se peut qu'il trouve une solution proche de l'optimale, mais parfois il se diverge vers des solutions pas assez bonnes, ceci est toujours causé par le choix de l'heuristique, une heuristique admissible peut assurer une solution optimale, mais ce n'est pas toujours le cas.

En revanche, les algorithmes DFS et BFS montrent des performances nettement moins satisfaisantes, avec des temps d'exécution et des nombres de nœuds générés exponentiellement croissants à mesure que le nombre de sacs augmente. Ces deux approches, bien que simples à implémenter et garantissant de trouver une solution optimale, souffrent de leur nature exploratoire exhaustive, ce qui les rend peu adaptées à des problèmes de taille importante en raison de leur complexité exponentielle.

En résumé, pour le problème du sac à dos multiple, l'algorithme A^* avec une heuristique bien choisie est clairement la meilleure option en termes de performances. Cependant, il est important de noter que le choix de l'heuristique peut avoir un impact significatif sur les performances d' A^* , et trouver une heuristique appropriée est souvent une tâche non triviale. D'où vient la nécessité d'explorer d'autres approches basées sur l'espace de solution comme les méta-heuristiques et les algorithmes évolutionnaires.